强化学习指南 2：单智能体 DQN 算法实践

一、实验目的

1. 理解为什么在连续状态空间中表格型 Q-learning 会失效，并掌握如何利用深度神经网络来近似 Q 价值函数。

2. 深入理解并实现 DQN 算法的核心基石：经验回放与目标网络。

3. 使用 PyTorch 搭建多层感知机（MLP），并完成神经网络的前向传播、损失计算与反向梯度更新。

4. 在 Gymnasium 环境库中，配置并使用连续状态离散动作环境 CartPole-v1，完成智能体训练。

二、实验环境

2.1 激活已有虚拟环境

打开终端或 Anaconda Prompt，激活上一节课创建的 rl_course 环境：

```
conda activate rl_course
```

2.2 安装及检查依赖库

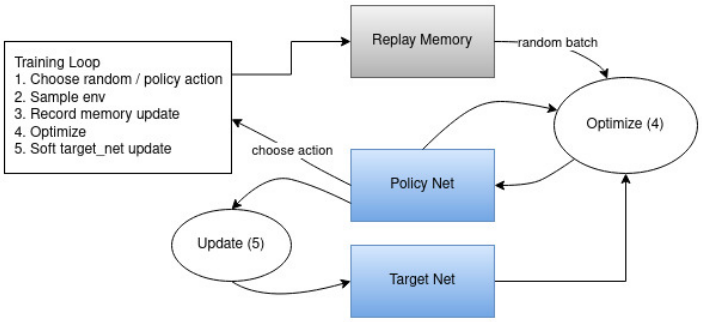
确保环境中成功安装了 gymnasium、numpy、matplotlib 以及 pytorch ，可通过以下 Python 代码进行验证：

```
import gymnasium as gym

import torch

print(torch.__version__) # 确认输出 Pytorch 版本号
```

三、任务说明

3.1 实验题目

基于连续状态空间 CartPole 环境的 DQN 求解实验



不同于 FrozenLake 的状态是有限的离散网格，倒立摆（CartPole）环境中，小车的位置、速度、杆的角度和角速度都是**连续值**，DQN 使用神经网络 $Q(s,a;\theta)$ 来替代 Q 表，通过输入状态 s，输出所有动作 a 的 Q 值，核心价值更新的目标公式为：

$$y = r + \gamma \max_{a'} Q(s',a';\theta^-)$$

其中 θ^- 为**目标网络**参数，目标网络不参与实时更新，提供稳定的目标预测值，作为标准答案来训练 Q 网络。

损失函数为当前网络预测值与目标值的均方误差：

$$L(\theta) = \mathrm{E}\left[\left(y - Q(s,a;\theta)\right)^2\right]$$

θ 为**当前 Q 网络**参数，通过反向传播缩小 Loss，训练神经网络。

PyTorch 官方 DQN 教程：
https://docs.pytorch.ac.cn/tutorials/intermediate/reinforcement_q_learning.html

3.2 实验要求：

3.2.1 实现单智能体 DQN 算法

包含以下模块：

- 经验回放池：实现数据的存储（State, Action, Reward, Next State, Done）与随机批量采样。
- 双网络结构：定义结构相同的 Policy Net（策略网络，用于动作选择和实时更新）与 Target Net（目标网络，用于计算**目标 Q 值**），实现目标网络的定期更新。
- epsilon-greedy 动作选择。
- 训练及测试闭环：实现智能体与环境的交互，完成交互-存储-采样-计算 Loss-网络参数更新的主干循环，并设计测试环节。

3.2.2 训练与测试要求

- 训练过程中需要记录每个回合的奖励总和。
- 训练结束后，关闭探索策略，将训练好的模型应用于**测试环境**，观察杆子是否能长时间保持直立不动。

3.2.3 可视化结果

- 绘制训练过程中的奖励曲线，注意设置滑动窗口让训练趋势更清晰。
- 成功率曲线。

- 测试可视化：**仅测试时**开启环境的渲染模式，展示训练结果。

```
env = gym.make('CartPole-v1', render_mode='human') # 渲染显示
```

3.2.4 建议参数范围

- 神经网络结构：隐藏层 1-2 层，每层 64 或 128 个神经元，激活函数为 ReLU。
- 学习率(alpha)：1e-4~1e-3
- 折扣因子(gamma)：0.98~0.99
- 经验回放池容量(Buffer Size)：10000~50000
- 批量大小(Batch Size)：32~128
- 目标网络更新频率：每 100~200 步更新一次（硬更新），或 $\tau = 0.005$（软更新）。
- 探索率(Epsilon)：初始值 1.0，衰减至最小值 0.01。
- 训练回合数：对于 CartPole 环境，通常 300~600 个回合即可收敛。